

OOPs with C++

Unit 2

C++ Classes and Objects

C++ is an object-oriented programming language.

Everything in C++ is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an **object**. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.

Attributes and methods are basically **variables** and **functions** that belongs to the class. These are often referred to as "class members".

A class is a user-defined data type that we can use in our program, and it works as an object constructor, or a "blueprint" for creating objects.

Create a Class

To create a class, use the **class** keyword:

Example

Create a class called "MyClass":

```
class MyClass {           // The class
public:                   // Access specifier
    int myNum;            // Attribute (int variable)
    string myString;      // Attribute (string variable)
};
```

Example explained

- The **class** keyword is used to create a class called **MyClass**.
- The **public** keyword is an **access specifier**, which specifies that members (attributes and methods) of the class are accessible from outside the class. You will learn more about [access specifiers](#) later.

- Inside the class, there is an integer variable `myNum` and a string variable `myString`. When variables are declared within a class, they are called **attributes**.
- At last, end the class definition with a semicolon `;`.

Create an Object

In C++, an object is created from a class. We have already created the class named `MyClass`, so now we can use this to create objects.

To create an object of `MyClass`, specify the class name, followed by the object name.

To access the class attributes (`myNum` and `myString`), use the dot syntax (`.`) on the object:

Example

Create an object called `myObj` and access the attributes:

```
class MyClass {           // The class
public:                   // Access specifier
    int myNum;            // Attribute (int variable)
    string myString;      // Attribute (string variable)
};

int main() {
    MyClass myObj;        // Create an object of MyClass

    // Access attributes and set values
    myObj.myNum = 15;
    myObj.myString = "Some text";

    // Print attribute values
    cout << myObj.myNum << "\n";
    cout << myObj.myString;
    return 0;
}
```

Multiple Objects

You can create multiple objects of one class:

Example

```
// Create a Car class with some attributes
class Car {
public:
    string brand;
    string model;
    int year;
};

int main() {
    // Create an object of Car
    Car carObj1;
    carObj1.brand = "BMW";
    carObj1.model = "X5";
    carObj1.year = 1999;

    // Create another object of Car
    Car carObj2;
    carObj2.brand = "Ford";
    carObj2.model = "Mustang";
    carObj2.year = 1969;

    // Print attribute values
    cout << carObj1.brand << " " << carObj1.model << " " <<
carObj1.year << "\n";
    cout << carObj2.brand << " " << carObj2.model << " " <<
carObj2.year << "\n";
    return 0;
}
```

C++ Access Specifiers

By now, you are quite familiar with the `public` keyword that appears in all of our class examples:

Example

```
class MyClass { // The class
    public:      // Access specifier
    // class members goes here
};
```

The **public** keyword is an **access specifier**. Access specifiers define how the members (attributes and methods) of a class can be accessed. In the example above, the members are **public** - which means that they can be accessed and modified from outside the code.

However, what if we want members to be private and hidden from the outside world?

In C++, there are three access specifiers:

- **public** - members are accessible from outside the class
- **private** - members cannot be accessed (or viewed) from outside the class
- **protected** - members cannot be accessed from outside the class, however, they can be accessed in inherited classes. You will learn more about [Inheritance](#) later.

In the following example, we demonstrate the differences between **public** and **private** members:

Example

```
class MyClass {
    public:    // Public access specifier
        int x; // Public attribute
    private:  // Private access specifier
        int y; // Private attribute
};

int main() {
    MyClass myObj;
    myObj.x = 25; // Allowed (public)
    myObj.y = 50; // Not allowed (private)
    return 0;
}
```

```
}
```

If you try to access a private member, an error occurs:

```
error: y is private
```

Note: It is possible to access private members of a class using a public method inside the same class. See the next chapter ([Encapsulation](#)) on how to do this.

Tip: It is considered good practice to declare your class attributes as private (as often as you can). This will reduce the possibility of yourself (or others) to mess up the code. This is also the main ingredient of the [Encapsulation](#) concept, which you will learn more about in the next chapter.

Note: By default, all members of a class are **private** if you don't specify an access specifier:

Example

```
class MyClass {  
    int x;    // Private attribute  
    int y;    // Private attribute  
};
```

C++ Functions

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

Functions are used to perform certain actions, and they are important for reusing code: Define the code once, and use it many times.

Create a Function

C++ provides some pre-defined functions, such as **main()**, which is used to execute code. But you can also create your own functions to perform certain actions.

To create (often referred to as *declare*) a function, specify the name of the function, followed by parentheses **()**:

Syntax

```
void myFunction() {  
    // code to be executed  
}
```

Example Explained

- **myFunction()** is the name of the function
- **void** means that the function does not have a return value. You will learn more about return values later in the next chapter
- inside the function (the body), add code that defines what the function should do

Call a Function

Declared functions are not executed immediately. They are "saved for later use", and will be executed later, when they are called.

To call a function, write the function's name followed by two parentheses **()** and a semicolon **;**

In the following example, **myFunction()** is used to print a text (the action), when it is called:

Example

Inside **main**, call **myFunction()**:

```
// Create a function  
void myFunction() {  
    cout << "I just got executed!";  
}  
  
int main() {  
    myFunction(); // call the function  
    return 0;  
}
```

```
}
```

```
// Outputs "I just got executed!"
```

A function can be called multiple times:

Example

```
void myFunction() {  
    cout << "I just got executed!\n";  
}
```

```
int main() {  
    myFunction();  
    myFunction();  
    myFunction();  
    return 0;  
}
```

```
// I just got executed!  
// I just got executed!  
// I just got executed!
```

Function Declaration and Definition

A C++ function consist of two parts:

- **Declaration:** the function's name, return type, and parameters (if any)
- **Definition:** the body of the function (code to be executed)

```
void myFunction() { // declaration  
    // the body of the function (definition)  
}
```

Note: If a user-defined function, such as `myFunction()` is declared after the `main()` function, **an error will occur**. It is because C++ works from top to bottom; which means that if the function is not declared above `main()`, the program is unaware of it:

Example

```
int main() {
    myFunction();
    return 0;
}

void myFunction() {
    cout << "I just got executed!";
}

// Error
```

However, it is possible to separate the declaration and the definition of the function - for code optimization.

You will often see C++ programs that have function declaration above `main()`, and function definition below `main()`. This will make the code better organized and easier to read:

Example

```
// Function declaration
void myFunction();

// The main method
int main() {
    myFunction(); // call the function
    return 0;
}

// Function definition
void myFunction() {
    cout << "I just got executed!";
}
```


C++ Function Parameters

Parameters and Arguments

Information can be passed to functions as a parameter. Parameters act as variables inside the function.

Parameters are specified after the function name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma:

Syntax

```
void functionName(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```

The following example has a function that takes a **string** called **fname** as parameter. When the function is called, we pass along a first name, which is used inside the function to print the full name:

Example

```
void myFunction(string fname) {  
    cout << fname << " Refsnes\n";  
}
```

```
int main() {  
    myFunction("Liam");  
    myFunction("Jenny");  
    myFunction("Anja");  
    return 0;  
}
```

```
// Liam Refsnes  
// Jenny Refsnes  
// Anja Refsnes
```

When a **parameter** is passed to the function, it is called an **argument**. So, from the example above: `fname` is a **parameter**, while `Liam`, `Jenny` and `Anja` are **arguments**.

Default Parameter Value

You can also use a default parameter value, by using the equals sign (=).

If we call the function without an argument, it uses the default value ("Norway"):

Example

```
void myFunction(string country = "Norway") {  
    cout << country << "\n";  
}
```

```
int main() {  
    myFunction("Sweden");  
    myFunction("India");  
    myFunction();  
    myFunction("USA");  
    return 0;  
}
```

```
// Sweden  
// India  
// Norway  
// USA
```

A parameter with a default value, is often known as an "**optional parameter**". From the example above, `country` is an optional parameter and `"Norway"` is the default value.

Multiple Parameters

Inside the function, you can add as many parameters as you want:

Example

```
void myFunction(string fname, int age) {
    cout << fname << " Refsnes. " << age << " years old. \n";
}

int main() {
    myFunction("Liam", 3);
    myFunction("Jenny", 14);
    myFunction("Anja", 30);
    return 0;
}

// Liam Refsnes. 3 years old.
// Jenny Refsnes. 14 years old.
// Anja Refsnes. 30 years old.
```

Note that when you are working with multiple parameters, the function call must have the same number of arguments as there are parameters, and the arguments must be passed in the same order.

Return Values

The `void` keyword, used in the previous examples, indicates that the function should not return a value. If you want the function to return a value, you can use a data type (such as `int`, `string`, etc.) instead of `void`, and use the `return` keyword inside the function:

Example

```
int myFunction(int x) {
    return 5 + x;
}

int main() {
    cout << myFunction(3);
    return 0;
}

// Outputs 8 (5 + 3)
```

This example returns the sum of a function with **two parameters**:

Example

```
int myFunction(int x, int y) {  
    return x + y;  
}  
  
int main() {  
    cout << myFunction(5, 3);  
    return 0;  
}  
  
// Outputs 8 (5 + 3)
```

You can also store the result in a variable:

Example

```
int myFunction(int x, int y) {  
    return x + y;  
}  
  
int main() {  
    int z = myFunction(5, 3);  
    cout << z;  
    return 0;  
}  
  
// Outputs 8 (5 + 3)
```

Pass By Reference

In the examples from the previous page, we used normal variables when we passed parameters to a function. You can also pass a [reference](#) to the function. This can be useful when you need to change the value of the arguments:

Example

```
void swapNums(int &x, int &y) {  
    int z = x;  
    x = y;  
}
```

```

    y = z;
}

int main() {
    int firstNum = 10;
    int secondNum = 20;

    cout << "Before swap: " << "\n";
    cout << firstNum << secondNum << "\n";

    // Call the function, which will change the values of firstNum and
    secondNum
    swapNums(firstNum, secondNum);

    cout << "After swap: " << "\n";
    cout << firstNum << secondNum << "\n";

    return 0;
}

```

C++ Function Overloading

With **function overloading**, multiple functions can have the same name with different parameters:

Example

```

int myFunction(int x)
float myFunction(float x)
double myFunction(double x, double y)

```

Consider the following example, which have two functions that add numbers of different type:

Example

```

int plusFuncInt(int x, int y) {
    return x + y;
}

double plusFuncDouble(double x, double y) {

```

```

    return x + y;
}

int main() {
    int myNum1 = plusFuncInt(8, 5);
    double myNum2 = plusFuncDouble(4.3, 6.26);
    cout << "Int: " << myNum1 << "\n";
    cout << "Double: " << myNum2;
    return 0;
}

```

Instead of defining two functions that should do the same thing, it is better to overload one.

In the example below, we overload the `plusFunc` function to work for both `int` and `double`:

Example

```

int plusFunc(int x, int y) {
    return x + y;
}

double plusFunc(double x, double y) {
    return x + y;
}

int main() {
    int myNum1 = plusFunc(8, 5);
    double myNum2 = plusFunc(4.3, 6.26);
    cout << "Int: " << myNum1 << "\n";
    cout << "Double: " << myNum2;
    return 0;
}

```

Note: Multiple functions can have the same name as long as the number and/or type of parameters are different.

cout

the `cout` object, together with the `<<` operator, is used to output values/print text:

Example

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!";
    return 0;
}
```

You can add as many `cout` objects as you want. However, note that it does not insert a new line at the end of the output:

Example

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!";
    cout << "I am learning C++";
    return 0;
}
```

cin

You have already learned that `cout` is used to output (print) values. Now we will use `cin` to get user input.

`cin` is a predefined variable that reads data from the keyboard with the extraction operator (`>>`).

In the following example, the user can input a number, which is stored in the variable `x`. Then we print the value of `x`:

Example

```
int x;
cout << "Type a number: "; // Type a number and press enter
```

```
cin >> x; // Get user input from the keyboard
cout << "Your number is: " << x; // Display the input value
```

Good To Know

cout is pronounced "see-out". Used for **output**, and uses the insertion operator (<<)

cin is pronounced "see-in". Used for **input**, and uses the extraction operator (>>)

Creating a Simple Calculator

In this example, the user must input two numbers. Then we print the sum by calculating (adding) the two numbers:

Example

```
int x, y;
int sum;
cout << "Type a number: ";
cin >> x;
cout << "Type another number: ";
cin >> y;
sum = x + y;
cout << "Sum is: " << sum;
```

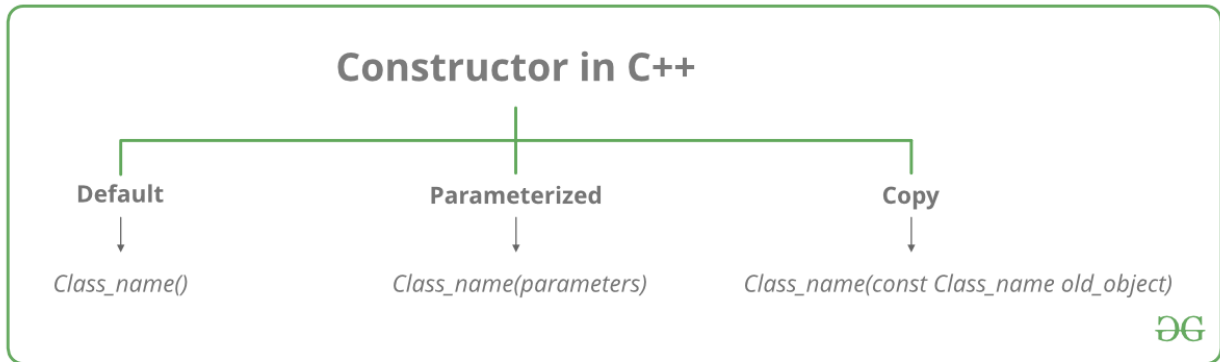
What is constructor?

A constructor is a member function of a class which initializes objects of a class. In C++, Constructor is automatically called when object(instance of class) create. It is special member function of the class.

How constructors are different from a normal member function?

A constructor is different from normal functions in following ways:

- Constructor has same name as the class itself
- Constructors don't have return type
- A constructor is automatically called when an object is created.
- If we do not specify a constructor, C++ compiler generates a default constructor for us (expects no parameters and has an empty body).



Types of Constructors

1. Default Constructors: Default constructor is the constructor which doesn't take any argument. It has no parameters.

- **CPP**

```
// Cpp program to illustrate the
// concept of Constructors
#include <iostream>
using namespace std;

class construct
{
public:
    int a, b;

    // Default Constructor
    construct()
    {
        a = 10;
        b = 20;
    }
};

int main()
{
    // Default constructor called
    // automatically
    // when the object is created
    construct c;
    cout << "a: " << c.a << endl
         << "b: " << c.b;
    return 1;
}
```

Output:

a: 10

b: 20

Note: Even if we do not define any constructor explicitly, the compiler will automatically provide a default constructor implicitly.

2. Parameterized Constructors: It is possible to pass arguments to constructors. Typically, these arguments help initialize an object when it is created. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object.

- **CPP**

```
// CPP program to illustrate
// parameterized constructors
#include <iostream>
using namespace std;

class Point
{
private:
    int x, y;

public:
    // Parameterized Constructor
    Point(int x1, int y1)
    {
        x = x1;
        y = y1;
    }

    int getX()
    {
        return x;
    }
    int getY()
    {
        return y;
    }
};

int main()
{
    // Constructor called
    Point p1(10, 15);
```

```

        // Access values assigned by
constructor
        cout << "p1.x = " << p1.getX() << ",
p1.y = " << p1.getY();

        return 0;
}

```

Output:

p1.x = 10, p1.y = 15

When an object is declared in a parameterized constructor, the initial values have to be passed as arguments to the constructor function. The normal way of object declaration may not work. The constructors can be called explicitly or implicitly.

Example e = Example(0, 50); // Explicit call

Example e(0, 50); // Implicit call

- **Uses of Parameterized constructor:**

1. It is used to initialize the various data elements of different objects with different values when they are created.
2. It is used to overload constructors.

- **Can we have more than one constructor in a class?**

Yes, It is called Constructor Overloading.

3. copy constructor

A copy constructor is a member function that initializes an object using another object of the same class. A copy constructor has the following general function prototype:

```
ClassName (const ClassName &old_obj);
```

Following is a simple example of copy constructor.

- **CPP**

```

#include<iostream>
using namespace std;

class Point
{
private:
    int x, y;
public:
    Point(int x1, int y1) { x = x1; y =
y1; }

    // Copy constructor
    Point(const Point &p1) {x = p1.x; y =
p1.y; }

```

```

        int getX()                { return x; }
        int getY()                { return y; }
};

int main()
{
    Point p1(10, 15); // Normal
    constructor is called here
    Point p2 = p1; // Copy constructor is
    called here

    // Let us access values assigned by
    constructors
    cout << "p1.x = " << p1.getX() << ",
p1.y = " << p1.getY();
    cout << "\np2.x = " << p2.getX() << ",
p2.y = " << p2.getY();

    return 0;
}

```

Output:

```

p1.x = 10, p1.y = 15
p2.x = 10, p2.y = 15

```

When is copy constructor called?

In C++, a Copy Constructor may be called in the following cases:

1. When an object of the class is returned by value.
2. When an object of the class is passed (to a function) by value as an argument.
3. When an object is constructed based on another object of the same class.
4. When the compiler generates a temporary object.

C++ Destructor

A destructor works opposite to constructor; it destructs the objects of classes. It can be defined only once in a class. Like constructors, it is invoked automatically.

A destructor is defined like constructor. It must have same name as class. But it is prefixed with a tilde sign (~).

Note: C++ destructor cannot have parameters. Moreover, modifiers can't be applied on destructors.

C++ Constructor and Destructor Example

Let's see an example of constructor and destructor in C++ which is called automatically.

```
1. #include <iostream>
2. using namespace std;
3. class Employee
4. {
5.     public:
6.         Employee()
7.         {
8.             cout<<"Constructor Invoked"<<endl;
9.         }
10.        ~Employee()
11.        {
12.            cout<<"Destructor Invoked"<<endl;
13.        }
14. };
15. int main(void)
16. {
17.     Employee e1; //creating an object of Employee
18.     Employee e2; //creating an object of Employee
19.     return 0;
20. }
```

Output:

```
Constructor Invoked
Constructor Invoked
Destructor Invoked
Destructor Invoked
```

C++ Dynamic Memory Allocation

A good understanding of how dynamic memory really works in C++ is essential to becoming a good C++ programmer. Memory in your C++ program is divided into two parts –

- **The stack** – All variables declared inside the function will take up memory from the stack.

- **The heap** – This is unused memory of the program and can be used to allocate the memory dynamically when program runs.

Many times, you are not aware in advance how much memory you will need to store particular information in a defined variable and the size of required memory can be determined at run time.

You can allocate memory at run time within the heap for the variable of a given type using a special operator in C++ which returns the address of the space allocated. This operator is called **new** operator.

If you are not in need of dynamically allocated memory anymore, you can use **delete** operator, which de-allocates memory that was previously allocated by new operator.

new and delete Operators

There is following generic syntax to use **new** operator to allocate memory dynamically for any data-type.

```
new data-type;
```

Here, **data-type** could be any built-in data type including an array or any user defined data types include class or structure. Let us start with built-in data types. For example we can define a pointer to type double and then request that the memory be allocated at execution time. We can do this using the **new** operator with the following statements –

```
double* pvalue = NULL; // Pointer initialized with null
pvalue = new double;   // Request memory for the variable
```

The memory may not have been allocated successfully, if the free store had been used up. So it is good practice to check if new operator is returning NULL pointer and take appropriate action as below –

```
double* pvalue = NULL;
if( !(pvalue = new double) ) {
    cout << "Error: out of memory." <<endl;
    exit(1);
}
```

The **malloc()** function from C, still exists in C++, but it is recommended to avoid using malloc() function. The main advantage of new over malloc() is that new doesn't just allocate memory, it constructs objects which is prime purpose of C++.

At any point, when you feel a variable that has been dynamically allocated is not anymore required, you can free up the memory that it occupies in the free store with the 'delete' operator as follows –

```
delete pvalue;           // Release memory pointed to by pvalue
```

Let us put above concepts and form the following example to show how 'new' and 'delete' work –

```

#include <iostream>
using namespace std;

int main () {
    double* pvalue = NULL; // Pointer initialized with null
    pvalue = new double;    // Request memory for the variable

    *pvalue = 29494.99;     // Store value at allocated address
    cout << "Value of pvalue : " << *pvalue << endl;

    delete pvalue;         // free up the memory.

    return 0;
}

```

If we compile and run above code, this would produce the following result –

Value of pvalue : 29495

Dynamic Memory Allocation for Arrays

Consider you want to allocate memory for an array of characters, i.e., string of 20 characters. Using the same syntax what we have used above we can allocate memory dynamically as shown below.

```

char* pvalue = NULL; // Pointer initialized with null
pvalue = new char[20]; // Request memory for the variable

```

To remove the array that we have just created the statement would look like this –

```

delete [] pvalue; // Delete array pointed to by
pvalue

```

Following the similar generic syntax of new operator, you can allocate for a multi-dimensional array as follows –

```

double** pvalue = NULL; // Pointer initialized with null
pvalue = new double [3][4]; // Allocate memory for a 3x4 array

```

However, the syntax to release the memory for multi-dimensional array will still remain same as above –

```

delete [] pvalue; // Delete array pointed to by
pvalue

```

Dynamic Memory Allocation for Objects

Objects are no different from simple data types. For example, consider the following code where we are going to use an array of objects to clarify the concept –

```

#include <iostream>
using namespace std;

```

```

class Box {
public:
    Box() {
        cout << "Constructor called!" <<endl;
    }
    ~Box() {
        cout << "Destructor called!" <<endl;
    }
};

int main() {
    Box* myBoxArray = new Box[4];
    delete [] myBoxArray; // Delete array

    return 0;
}

```

If you were to allocate an array of four Box objects, the Simple constructor would be called four times and similarly while deleting these objects, destructor will also be called same number of times.

If we compile and run above code, this would produce the following result –

```

Constructor called!
Constructor called!
Constructor called!
Constructor called!
Destructor called!
Destructor called!
Destructor called!
Destructor called!

```

C++ Friend Functions

A friend function of a class is defined outside that class' scope but it has the right to access all private and protected members of the class. Even though the prototypes for friend functions appear in the class definition, friends are not member functions.

A friend can be a function, function template, or member function, or a class or class template, in which case the entire class and all of its members are friends.

To declare a function as a friend of a class, precede the function prototype in the class definition with keyword **friend** as follows –

```

class Box {
    double width;

public:
    double length;
    friend void printWidth( Box box );
}

```



```
        void setWidth( double wid );
};
```

To declare all member functions of class ClassTwo as friends of class ClassOne, place a following declaration in the definition of class ClassOne –

```
friend class ClassTwo;
```

Consider the following program –

```
#include <iostream>

using namespace std;

class Box {
    double width;

public:
    friend void printWidth( Box box );
    void setWidth( double wid );
};

// Member function definition
void Box::setWidth( double wid ) {
    width = wid;
}

// Note: printWidth() is not a member function of any class.
void printWidth( Box box ) {
    /* Because printWidth() is a friend of Box, it can
       directly access any member of this class */
    cout << "Width of box : " << box.width << endl;
}

// Main function for the program
int main() {
    Box box;

    // set box width without member function
    box.setWidth(10.0);

    // Use friend function to print the width.
    printWidth( box );

    return 0;
}
```

When the above code is compiled and executed, it produces the following result –

Width of box : 10

C++ this Pointer

Every object in C++ has access to its own address through an important pointer called **this** pointer. The **this** pointer is an implicit parameter to all member functions. Therefore, inside a member function, this may be used to refer to the invoking object.

Friend functions do not have a **this** pointer, because friends are not members of a class. Only member functions have a **this** pointer.

Let us try the following example to understand the concept of this pointer –

```
#include <iostream>

using namespace std;

class Box {
public:
    // Constructor definition
    Box(double l = 2.0, double b = 2.0, double h = 2.0) {
        cout << "Constructor called." << endl;
        length = l;
        breadth = b;
        height = h;
    }
    double Volume() {
        return length * breadth * height;
    }
    int compare(Box box) {
        return this->Volume() > box.Volume();
    }

private:
    double length;    // Length of a box
    double breadth;    // Breadth of a box
    double height;    // Height of a box
};

int main(void) {
    Box Box1(3.3, 1.2, 1.5);    // Declare box1
    Box Box2(8.5, 6.0, 2.0);    // Declare box2

    if(Box1.compare(Box2)) {
        cout << "Box2 is smaller than Box1" << endl;
    } else {
        cout << "Box2 is equal to or larger than Box1" << endl;
    }
}
```

```
}  
  
    return 0;  
}
```

When the above code is compiled and executed, it produces the following result –

Constructor called.

Constructor called.

Box2 is equal to or larger than Box1